Searching & Sorting Tutorial Solutions

Searching

Q.1) (b)

x	y	.	.	.	z
i	j				

So, if condition will check a[j] - a[i] < S then increment i so that difference between two elements increase and when a[j] - a[i] > S increment i for making difference between two elements indexed at i and j respectively is decrease. So, a[j] - a[i] < S is correct condition.

Q.2) (d)

	4	6	8	10	12	15	**18**	12	8	7
a	1	2	3	4	5	6	7	8	9	10

Using Binary Search:
First we need to find the peak in array using following pseudocode:

```
low = 1; high = n;
while(low <= high) {
        m = (low + high)/2;
        if (a[mid - 1] < a[mid] < a[mid + 1])
                low = mid + 1;
        else if [mid - 1] > a[mid] > a[mid + 1]
                high = mid-1;
        else if (a[mid - 1] > a[mid] > a[mid + 1])
                return(mid);
}
```

After finding peak we can apply binary search to left and right arrays w.r.t peak found
Therefore Total Complexity = $\Theta(\log n)+\Theta(\log n)+\Theta(\log n) = 3\Theta(\log n) = \Theta(\log n)$

Q.3) (c) The given program doesn't work for the cases where element to be searched is the last element of Y[] or greater than the last element (or maximum element) in Y[]. For such cases, program goes in an infinite loop because i is assigned value as k in all iterations, and i never becomes equal to or greater than j. So while condition never becomes false.

Q.4) (c) For I) When a[i] >= b[i] then 2i elements smaller than equal to a[i]. Since 'i' elements to array f that are less than equal to bi and 'i' element of array 'a' which are less than equal to a[i]. So, a[i] will come after c[2i] i.e. c[2i] <= a[i] is true.

For II) When a[i] >= b[i] then b[i] always be less than equal to c[2i] i.e. when 'i' elements of array 'a' and 'i' elements of array 'b' are same then maximum index of b[i] is c[2i]. So, c[2i] >= b[i] is true.

Answer can be concluded form these two observations.

Q.5) (a) The usual O(n ^ 2) implementation of insertion sort to sort an array uses binary search to identify remain O(n ^ 2) The time required to insert an the position, the worst case running time will element x into a binary search tree is bounded made between x and elements already in the tree by a constant times the number of comparisons Thus we can measure time in terms of the number of comparisons made. In worst case implementation of insertion sort takes O(n ^ 2) So in the worst case adding n elements to a binary search tree could require O(n ^ 2) time. In expected case means element are sorted in increasing order adding n elements to a binary search tree could require O(n log n) time.

Q.6) (c) In selection sort, minimum number of swaps taken compared to other sorting techniques.
Number of swaps = O(n)
Number of comparisons = $O(n^2)$
In finds the minimum of the subarray and exchanges only once in every pass.

Q.7) Ans = 3 , A merge is **void** when every element of the left sub-array is already ≤ every element of the right sub-array, so the merge simply concatenates them with no interleaving.

Trace merge sort on $A = [10,7,8,19,41,35,25,31]$:

Bottom level (pairs):
$[10], [7] \rightarrow [7,10]$ (result is R then L, not "L then R") — **not void**.
$[8], [19] \rightarrow [8,19]$ (already L then R as-is) — **void**.
$[41], [35] \rightarrow [35,41]$ — **not void**.
$[25], [31] \rightarrow [25,31]$ (already in order) — **void**.

Next level (quads):
$[7,10] + [8,19] \rightarrow [7,8,10,19]$ (interleaved, not simply L then R) — **not void**.
$[35,41] + [25,31] \rightarrow [25,31,35,41]$ (interleaved) — **not void**.

Final merge:
$[7,8,10,19] + [25,31,35,41]$: every element of the left (≤ 19) is already smaller than every element of the right (≥ 25), so the merge is simply L followed by R — **void**.

Total merges $= 4 + 2 + 1 = 7$ (matches the given count), with exactly 3 of them void.

Q.8) Ans = 10. All swaps are in following order:

8,7,22,9,31,5,13
8,7,9,22,31,5,13
8,7,9,22,5,31,13
8,7,9,22,5,13,31
7,8,9,22,5,13,31
7,8,9,5,22,13,31
7,8,9,5,13,22,31
7,8,5,9,13,22,31
7,5,8,9,13,22,31
5,7,8,9,13,22,31

Q.9) (c)
There are two cases, when Randomized Quick Sort will result into worst case time complexity of $O(n^2)$

1. When all elements are same in the input array, Partition algorithm will divide input array in two sub-arrays, one with $n - 1$ elements and second with 0 element. There is an assumption here that, we are using the same partition algorithm without any modification.

2. If the randomised pivot selector happens to select the smallest or largest element N times in a row, we will get the worst possible performance.

Q.10) (b)

